

ACM India Summer School on Symmetric Key Cryptography

Day 6: Analysis and Implementation of the Salsa20 Stream Cipher

Sourav Sharma

June 13, 2026

1 Introduction

Salsa20 is a high-speed stream cipher designed by Daniel J. Bernstein, based on the Add-Rotate-XOR (ARX) operations. The cipher operates on a state of sixteen 32-bit words (512 bits in total), arranged in a 4×4 grid. In this lab, I implemented the core Salsa20 transformations, encryption/decryption mechanisms, and studied its differential propagation properties.

2 Salsa20 Core Functions

2.1 Quarter Round (QR)

The Quarter Round function is the fundamental ARX building block of Salsa20. It operates on four 32-bit words (a, b, c, d) and updates them in place:

$$\begin{aligned} b &\leftarrow b \oplus \text{rotl}(a + d, 7) \\ c &\leftarrow c \oplus \text{rotl}(b + a, 9) \\ d &\leftarrow d \oplus \text{rotl}(c + b, 13) \\ a &\leftarrow a \oplus \text{rotl}(d + c, 18) \end{aligned}$$

where $\text{rotl}(x, n)$ represents a left circular shift of x by n bits.

2.2 Row, Column, and Double Rounds

Salsa20 alternates rounds of column and row transformations. A Column Round applies the QR function to the four columns of the state grid:

$$\text{qr}(0, 4, 8, 12), \quad \text{qr}(5, 9, 13, 1), \quad \text{qr}(10, 14, 2, 6), \quad \text{qr}(15, 3, 7, 11)$$

A Row Round applies the QR function to the rows (with diagonal starting points):

$$\text{qr}(0, 1, 2, 3), \quad \text{qr}(5, 6, 7, 4), \quad \text{qr}(10, 11, 8, 9), \quad \text{qr}(15, 12, 13, 14)$$

A Double Round is defined as one Column Round followed by one Row Round. Salsa20/20 executes 10 double rounds (20 rounds total).

3 Salsa20 Encryption and Decryption

The 512-bit state matrix is initialized with four constant words, an 8-word (256-bit) key, a 2-word (64-bit) nonce, and a 2-word (64-bit) block counter:

$$\begin{pmatrix} \text{const}_0 & \text{key}_0 & \text{key}_1 & \text{key}_2 \\ \text{key}_3 & \text{const}_1 & \text{nonce}_0 & \text{nonce}_1 \\ \text{ctr}_0 & \text{ctr}_1 & \text{const}_2 & \text{key}_4 \\ \text{key}_5 & \text{key}_6 & \text{key}_7 & \text{const}_3 \end{pmatrix}$$

The keystream block is generated by running 10 double rounds on the state and then adding the original input state word-by-word to the final round output (feed-forward step to ensure invertibility is impossible without the key). Encryption and decryption are symmetric; both are performed by XORing the plaintext/ciphertext with the generated keystream. I verified my implementation (`salsa_enc_dec.cpp`) on a test string, achieving full decryption recovery.

4 Differential Cryptanalysis

4.1 Hamming Weight Propagation after One Round

We studied the propagation of a single-bit input difference at $\Delta X = X_7[31]$ (MSB of the 8th word) through 1 column round (which is the first round of a double round). Because columns 0, 1, and 2 are inactive (having zero difference), only Column 3 (indices 15, 3, 7, 11) is active:

1. $b \leftarrow 3, a \leftarrow 15, d \leftarrow 11$: Since X_{15} and X_{11} have zero difference, X_3 receives zero difference.
2. $c \leftarrow 7, b \leftarrow 3, a \leftarrow 15$: Since X_3 and X_{15} have zero difference, the addition/rotation yields zero difference. Thus, X_7 retains its original input difference: $\Delta X_7 = 2^{31}$ (Hamming weight = 1).
3. $d \leftarrow 11, c \leftarrow 7, b \leftarrow 3$: Here we compute $\text{rotr}(X_7 + X_3, 13)$. Since the input difference is at the MSB of X_7 (2^{31}), addition modulo 2^{32} preserves the MSB difference exactly (no carry is possible beyond the 31st bit). Thus, the difference entering the rotation is exactly 2^{31} . Rotating left by 13 bits gives a difference at bit 12: $\Delta X_{11} = 2^{12}$ (Hamming weight = 1).
4. $a \leftarrow 15, d \leftarrow 11, c \leftarrow 7$: We compute $\text{rotr}(X_{11} + X_7, 18)$. The input difference in X_{11} is at bit 12, which propagates due to carries in addition, resulting in an average carry chain length of 2 (affecting bits 12, 13, etc.). The difference in X_7 is at bit 31. Therefore, the sum difference has an average Hamming weight of ≈ 3 active bits. Rotating this by 18 bits preserves the Hamming weight. Thus, ΔX_{15} receives an average difference weight of ≈ 3 .

Adding these active differences, the total average Hamming weight of the state difference after 1 round is:

$$\text{HW}(\Delta Y) = \text{HW}(\Delta X_7) + \text{HW}(\Delta X_{11}) + \text{HW}(\Delta X_{15}) \approx 1 + 1 + 3 = 5$$

This theoretical prediction is verified empirically by my simulation in `Salsa20.cpp`, which reports an average Hamming weight of exactly **4.985** over 100,000 random samples.

4.2 Differential Probability of Salsa20/4

We estimated the differential probability for the transition $\Delta X = X_7[31] \rightarrow \Delta Y = Y_1[14]$ after 4 rounds (Salsa20/4). Over 2^{22} simulated random pairs, the probability of no difference ($\Delta Y_1[14] = 0$) was measured to be approximately **0.5412**. This shows a significant bias away from the uniform distribution value of 0.50, presenting a potential distinguisher.